The Equations of Holonomic Unified Field Dynamics

Symbol Glossary, Progressive Expansion, and Implementation

Reference Document for the Live Equation Rewriter

February 2026

This document presents each core equation of the HUFD framework in a uniform structure. For every equation: first the compact symbolic form appears, then every symbol is described in plain language. Next, the formula is expanded one layer at a time, with every new symbol that appears described at the expansion in which it first occurs. This continues until the expression is reduced to primitive operations. Finally, the computational implementation is given in Python. Equation numbers in parentheses reference the main HUFD paper by J. Harlow (Nov. 2025).

Contents

1 The HUFD Action Functional S_{HUFD} **3**
- 1.1 Compact Form (Eq. a.1) . . . 3
- 1.2 Symbol Descriptions . . . 3
- 1.3 First Expansion — Curvature Term . . . 3
- 1.4 Second Expansion — Lucidity Functional . . . 4
- 1.5 Third Expansion — Drift Energy and Phase-Lock . . . 4
- 1.6 The Action Fully Assembled . . . 5
- 1.7 Implementation . . . 5

2 The Curvature Tensor $R_{\mu\nu}$ **7**
- 2.1 Compact Form (Eq. 2.1) . . . 7
- 2.2 Symbol Descriptions . . . 7
- 2.3 First Expansion — Commutator . . . 7
- 2.4 Second Expansion — Connection in the $\mathfrak{su}(2)$ Basis . . . 7
- 2.5 Third Expansion — Curvature in Components . . . 8
- 2.6 Implementation . . . 8

3 The Charlton Metric and Observer Projection **9**
- 3.1 Compact Forms . . . 9
- 3.2 Symbol Descriptions . . . 9
- 3.3 Expansion — Projector Acting on Curvature . . . 9
- 3.4 Implementation . . . 9

4 The Field Equation — Gradient Flow **11**
- 4.1 Compact Form (Eq. 28) . . . 11
- 4.2 Symbol Descriptions . . . 11
- 4.3 First Expansion (Eq. 5.2) . . . 11
- 4.4 Second Expansion — Lattice Discretisation (Eq. 29) . . . 11
- 4.5 Third Expansion — Fully Explicit Lattice Step . . . 12
- 4.6 Implementation . . . 12

5 The Lyapunov Stability Guarantee **14**

5.1 Compact Form . . . 14
5.2 Symbol Descriptions . . . 14
5.3 Expansion — Why It Holds . . . 14
5.4 Implementation . . . 14

6 The Gauge Symmetry **15**
6.1 Compact Form (Eq. 3.1) . . . 15
6.2 Symbol Descriptions . . . 15
6.3 Expansion — Curvature Transforms Covariantly . . . 15
6.4 Implementation . . . 15

7 The Constant-Attention Update **17**
7.1 Compact Form (Eq. 2.2) . . . 17
7.2 Symbol Descriptions . . . 17
7.3 Expansion — The Constant-Attention Loop . . . 17
7.4 Implementation . . . 17

8 Lattice Holonomy and Discrete Curvature **19**
8.1 Compact Form . . . 19
8.2 Symbol Descriptions . . . 19
8.3 Expansion — Quaternion Product . . . 19
8.4 Curvature from Holonomy . . . 19
8.5 Relaxation — SLERP Correction . . . 20
8.6 Implementation . . . 20

9 The Observables **22**
9.1 Attractor Entropy (Eq. A6) . . . 22
9.2 Total Curvature Energy . . . 22
9.3 Lucidity Flux . . . 22
9.4 Implementation . . . 22

10 Summary of Rewrite Rules **24**

The HUFD Action Functional $S_{\mathbf{HUFD}}$

Compact Form (Eq. a.1)

$$S_{\mathrm{HUFD}} \;=\; \int d^4x \;\mathrm{Tr}\Big(R_{\mu\nu}\,R^{\mu\nu} \;+\; \alpha\,L_{\mathrm{sem}} \;+\; \beta\,\mathcal{E}_d \;+\; \gamma\,\Phi^{-1}\Big) \tag{1}$$

Symbol Descriptions

The symbol S_{HUFD} (the HUFD action) is a single scalar number summarising the total state of the system. Every dynamical law in the framework derives from requiring this number to be stationary under small variations of the field.

The integral $\int d^4x$ runs over all four dimensions of the manifold—three spatial and one temporal. Every point in the manifold contributes to the total action.

The operator Tr (trace) sums the diagonal entries of the matrix-valued expression inside. Because $\mathcal{H}_\mu$ takes values in the Lie algebra $\mathfrak{su}(2)$, every term inside the integral is a 2×2 matrix; the trace converts it to a scalar.

The tensor $R_{\mu\nu}$ (curvature) measures how the field connection $\mathcal{H}_\mu$ fails to commute across different directions. It is the central geometric object of the theory. Its contraction $R_{\mu\nu}\,R^{\mu\nu}$ is the curvature energy density—the "self-interaction" of the field.

The indices μ and ν are spacetime indices running over the four dimensions of the manifold. The raised index $R^{\mu\nu}$ is obtained by contracting with the inverse Charlton metric $\eta^{\mu\nu}$.

The scalar α (alpha) is a dimensionless coupling coefficient weighting the relative importance of semantic coherence in the action. Similarly β (beta) weights the drift energy and γ (gamma) weights the inverse phase-lock.

The functional L_{sem} (semantic lucidity) quantifies local smoothness and global stability of the semantic potential relative to its archetypal baseline. Lower values correspond to higher conceptual coherence.

The functional $\mathcal{E}_d$ (drift energy) measures the temporal divergence of the holonomic state—how fast the field is changing. Lower drift means more stable recurrent behaviour under Constant Attention.

The scalar Φ (phase-lock index) represents the degree of coherence between successive field states. $\Phi \to 1$ indicates full holonomic phase-locking and steady cognitive resonance. The inverse Φ^{-1} appears in the action so that decreasing phase-lock increases the action, penalising incoherence.

First Expansion — Curvature Term

Replacing $R_{\mu\nu}$ with its definition (Eq. 2.1):

$$S_{\mathrm{HUFD}} = \int d^4x \;\mathrm{Tr}\Big([\mathcal{H}_\mu,\,\mathcal{H}_\nu]\,[\mathcal{H}^\mu,\,\mathcal{H}^\nu] + \omega_{[\mu}\,\eta_{\nu]}\,\omega^{[\mu}\,\eta^{\nu]} + \text{cross terms} + \alpha\,L_{\mathrm{sem}} + \beta\,\mathcal{E}_d + \gamma\,\Phi^{-1}\Big) \tag{2}$$

The commutator bracket $[\mathcal{H}_\mu,\,\mathcal{H}_\nu] = \mathcal{H}_\mu\,\mathcal{H}_\nu - \mathcal{H}_\nu\,\mathcal{H}_\mu$ (the Lie bracket) computes the failure of commutativity between two components of the connection. Because $\mathcal{H}_\mu$ takes values in $\mathfrak{su}(2)$, this product is matrix multiplication of 2×2 traceless anti-Hermitian matrices.

The symbol $\mathcal{H}_\mu$ (the holonomic field connection) is the primary dynamical variable of the entire theory. It is an $\mathfrak{su}(2)$-valued one-form on the manifold, governing the internal parallel transport of semantic states from one point to another.

The one-form ω_μ (omega, the observer congruence) defines the temporal foliation—the reference frame of the observer. It picks out the "time direction" in the manifold.

The antisymmetrisation $\omega_{[\mu}\,\eta_{\nu]} = \frac{1}{2}(\omega_\mu\,\eta_\nu - \omega_\nu\,\eta_\mu)$ is the background curvature contribution from Charlton Geometry, coupling the observer congruence to the metric.

The tensor $\eta_{\mu\nu}$ (eta, the Charlton metric) encodes the geometry of the observer-split manifold. It is not the flat Minkowski metric; it is defined below.

Second Expansion — Lucidity Functional

Replacing L_{sem} with its definition (Eq. 4.1):

$$L_{\text{sem}} \;=\; \int_{\mathcal{M}} \Big(\big\|\nabla_\mu\,\varphi\big\|_g^{\,2} \;+\; \lambda\,\big\|\varphi - \varphi^*\big\|_g^{\,2} \Big)\, d\text{vol}_g \tag{3}$$

The manifold $\mathcal{M}$ (the semantic manifold) is a Riemannian space whose points represent meaning-states. It is equipped with a metric g induced by co-occurrence and entailment statistics.

The field φ (phi, the semantic potential) is a scalar field on $\mathcal{M}$ whose value at each point encodes the local semantic content—the "meaning" at that location.

The field φ^* (phi-star, the archetypal baseline) is the equilibrium configuration of the semantic potential. It represents the ideal, fully coherent distribution of meaning. The difference $\varphi - \varphi^*$ measures departure from coherence.

The covariant derivative $\nabla_\mu\,\varphi$ measures how the semantic potential changes from point to point on the manifold, accounting for the curvature of the underlying space.

The norm $\|\cdot\|_g$ is computed using the metric g on $\mathcal{M}$. The squared gradient $\|\nabla_\mu\,\varphi\|_g^2$ measures the roughness of the semantic field—rapid spatial variation of meaning.

The coefficient λ (lambda) controls the strength of the restoring force pulling φ toward φ^*. Larger λ means the system more strongly penalises departure from the archetypal baseline.

The volume form $d\text{vol}_g = \sqrt{\det g}\;d^n x$ ensures the integral is invariant under coordinate changes on $\mathcal{M}$.

Third Expansion — Drift Energy and Phase-Lock

Replacing $\mathcal{E}_d$ with its definition (Eq. 4.2) and Φ with its definition (Eq. 4.3):

$$\mathcal{E}_d \;=\; \big\|\nabla_t\,\psi\big\|^2 \tag{4}$$

$$\Phi \;=\; \big|\langle \psi_t,\; \psi_{t+\tau}\rangle\big| \tag{5}$$

The field ψ (psi, the holonomic state) is the full state vector being transported through the field. It encodes the current cognitive or semantic content propagating under the connection $\mathcal{H}_\mu$.

The temporal covariant derivative $\nabla_t\,\psi = \partial_t\,\psi + \mathcal{H}_t\,\psi$ measures the rate of change of the state along the time direction, accounting for parallel transport. Its squared norm is the drift energy.

The inner product $\langle \psi_t, \psi_{t+\tau} \rangle$ measures the overlap between the state at time t and the state at a later time $t + \tau$. The absolute value $|\cdot|$ gives a real number between 0 and 1.

The time lag τ (tau) is the separation over which phase coherence is measured.

When $\Phi = 1$, the state at t and $t + \tau$ are perfectly aligned—full phase-lock. When $\Phi \to 0$, the states are orthogonal—complete decoherence.

The Action Fully Assembled

Substituting all expansions back into the action:

$$\boxed{\begin{aligned} S_{\text{HUFD}} = \int d^4x \ \mathrm{Tr}\Big(& [\mathcal{H}_\mu, \mathcal{H}_\nu]\,[\mathcal{H}^\mu, \mathcal{H}^\nu] + \omega_{[\mu}\eta_{\nu]}\omega^{[\mu}\eta^{\nu]} \\ & + \alpha \int_{\mathcal{M}} \Big(\|\nabla_\mu \varphi\|_g^2 + \lambda \|\varphi - \varphi^*\|_g^2 \Big) d\mathrm{vol}_g \\ & + \beta \|\nabla_t \psi\|^2 + \gamma \big|\langle \psi_t, \psi_{t+\tau} \rangle\big|^{-1} \Big) \end{aligned}} \tag{6}$$

Every symbol in this expression has now been defined. The expression is a scalar-valued integral over spacetime of a trace of matrix- and scalar-valued terms.

Implementation

```
import numpy as np

def hufd_action(H, omega, eta, phi, phi_star, psi, psi_lagged,
                alpha, beta, gamma, lam, dx=1.0):
    """
    Evaluate the HUFD action on a discrete lattice.

    H         : array [N, N, 4, 2, 2]  -- connection, mu,nu indices,
        su(2) matrices
    omega     : array [N, 4]            -- observer congruence 1-form
    eta       : array [N, 4, 4]         -- Charlton metric at each
        node
    phi       : array [N]               -- semantic potential
    phi_star  : array [N]               -- archetypal baseline
    psi       : array [N, d]            -- holonomic state at time t
    psi_lagged: array [N, d]            -- holonomic state at time t +
         tau
    """
    N = H.shape[0]

    # --- Curvature energy: Tr(R_munu R^munu) ---
    curv_energy = 0.0
    for mu in range(4):
        for nu in range(mu+1, 4):
            # Commutator [H_mu, H_nu] at each node
            comm = H[:, mu] @ H[:, nu] - H[:, nu] @ H[:, mu]
            # Background term omega_[mu eta_nu]
            bg = 0.5 * (np.outer(omega[:, mu], eta[:, nu])
                      - np.outer(omega[:, nu], eta[:, mu]))
            R = comm  # + bg contribution (simplified)
```

```
            curv_energy += np.sum(np.trace(R @ R, axis1=-2, axis2=-1)
                )

    # --- Lucidity: ||grad phi||^2 + lambda ||phi - phi*||^2 ---
    grad_phi = np.gradient(phi)  # finite-difference approximation
    L_sem = np.sum(grad_phi**2) + lam * np.sum((phi - phi_star)**2)

    # --- Drift energy: ||d_t psi||^2 ---
    dpsi_dt = psi_lagged - psi  # finite-difference in time
    E_d = np.sum(dpsi_dt**2)

    # --- Phase-lock: |<psi_t, psi_{t+tau}>| ---
    overlap = np.abs(np.sum(psi * psi_lagged, axis=-1))
    Phi = np.mean(overlap)
    Phi_inv = 1.0 / (Phi + 1e-8)  # regularised inverse

    # --- Total action ---
    S = (curv_energy + alpha * L_sem + beta * E_d + gamma * Phi_inv)
       * dx**4
    return S
```

The Curvature Tensor $R_{\mu\nu}$

Compact Form (Eq. 2.1)

$$R_{\mu\nu} \;=\; [\mathcal{H}_\mu,\, \mathcal{H}_\nu] \;+\; \omega_{[\mu}\,\eta_{\nu]} \tag{7}$$

Symbol Descriptions

The tensor $R_{\mu\nu}$ (the curvature) is an $\mathfrak{su}(2)$-valued antisymmetric tensor. It captures the total "bending" of the holonomic field. Both indices μ and ν are spacetime indices. Antisymmetry means $R_{\mu\nu} = -R_{\nu\mu}$.

The commutator $[\mathcal{H}_\mu,\, \mathcal{H}_\nu]$ is the non-abelian part of the curvature—the pure field self-interaction. If $\mathcal{H}_\mu$ were an ordinary number (abelian), this would vanish. It is nonzero precisely because the gauge group SU(2) is non-abelian.

The background term $\omega_{[\mu}\,\eta_{\nu]}$ couples the observer congruence ω to the Charlton metric η. This term has no analogue in standard Yang–Mills theory; it arises from the observer-split structure of Charlton Geometry.

First Expansion — Commutator

$$R_{\mu\nu} \;=\; \mathcal{H}_\mu\,\mathcal{H}_\nu \;-\; \mathcal{H}_\nu\,\mathcal{H}_\mu \;+\; \frac{1}{2}(\omega_\mu\,\eta_\nu - \omega_\nu\,\eta_\mu) \tag{8}$$

The products $\mathcal{H}_\mu\,\mathcal{H}_\nu$ and $\mathcal{H}_\nu\,\mathcal{H}_\mu$ are matrix multiplications in $\mathfrak{su}(2)$. Each component $\mathcal{H}_\mu$ is a 2×2 traceless anti-Hermitian matrix. Their product is a 2×2 matrix; their difference is the commutator.

The factor $\frac{1}{2}$ arises from the antisymmetrisation convention $A_{[\mu}\,B_{\nu]} = \frac{1}{2}(A_\mu\,B_\nu - A_\nu\,B_\mu)$.

Second Expansion — Connection in the $\mathfrak{su}(2)$ Basis

Each component of the connection can be expanded in the Pauli basis:

$$\mathcal{H}_\mu \;=\; \mathcal{H}_\mu^{\,a}\,\frac{i\,\sigma_a}{2} \;=\; \mathcal{H}_\mu^{\,1}\,\frac{i\,\sigma_1}{2} \;+\; \mathcal{H}_\mu^{\,2}\,\frac{i\,\sigma_2}{2} \;+\; \mathcal{H}_\mu^{\,3}\,\frac{i\,\sigma_3}{2} \tag{9}$$

The index a is a Lie-algebra index running over $\{1, 2, 3\}$, the three generators of SU(2).

The matrices σ_a (the Pauli matrices) are the standard 2×2 generators:

$$\sigma_1 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad \sigma_2 = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad \sigma_3 = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \tag{10}$$

The factor $i/2$ is the conventional normalisation ensuring that the Lie-algebra commutation relations take the form $[i\sigma_a/2,\; i\sigma_b/2] = -\epsilon_{abc}\; i\sigma_c/2$, where ϵ_{abc} (the Levi-Civita symbol) equals $+1$ for even permutations of $(1, 2, 3)$, -1 for odd, and 0 if any indices repeat.

The real coefficients $\mathcal{H}_\mu^{\,a}$ are three numbers per spacetime direction—twelve real degrees of freedom in total for a four-dimensional manifold. These are the actual numerical quantities stored at each point.

Third Expansion — Curvature in Components

Substituting the Pauli expansion into the commutator:

$$[\mathcal{H}_\mu, \mathcal{H}_\nu] \;=\; -\,\epsilon_{abc}\,\mathcal{H}^a_\mu\,\mathcal{H}^b_\nu\,\frac{i\,\sigma_c}{2} \tag{11}$$

This shows that the curvature commutator produces a new $\mathfrak{su}(2)$ element whose c-th component is $-\epsilon_{abc}\,\mathcal{H}^a_\mu\,\mathcal{H}^b_\nu$, summed over a and b (Einstein summation convention). The structure constants ϵ_{abc} encode the non-commutativity of the gauge group.

Implementation

```
import numpy as np

# Pauli matrices (as complex 2x2)
sigma = np.array([
    [[0, 1], [1, 0]],         # sigma_1
    [[0, -1j], [1j, 0]],      # sigma_2
    [[1, 0], [0, -1]],        # sigma_3
], dtype=complex)

def connection_matrix(H_components):
    """
    Build su(2) matrix from 3 real components.
    H_components : array of shape (3,) -- [H^1, H^2, H^3]
    Returns      : array of shape (2,2) -- the su(2) matrix
    """
    return sum(H_components[a] * 1j * sigma[a] / 2 for a in range(3))

def curvature(H_mu, H_nu, omega_mu, eta_nu, omega_nu, eta_mu):
    """
    Compute R_{mu,nu} = [H_mu, H_nu] + omega_[mu eta_nu].

    H_mu, H_nu : arrays of shape (2,2) -- su(2) connection matrices
    omega, eta : scalar values at given mu, nu components
    Returns    : array of shape (2,2) -- the curvature matrix
    """
    commutator = H_mu @ H_nu - H_nu @ H_mu
    background = 0.5 * (omega_mu * eta_nu - omega_nu * eta_mu)
    R = commutator + background * np.eye(2)
    return R
```

The Charlton Metric and Observer Projection

Compact Forms

$$\eta_{\mu\nu} \;=\; \delta_{\mu\nu} \;-\; 2\,\omega_\mu \otimes \omega_\nu \tag{12}$$

$$\Pi_{\mu\nu} \;=\; \delta_{\mu\nu} \;-\; \omega_\mu \otimes \omega_\nu \tag{13}$$

Symbol Descriptions

The tensor $\eta_{\mu\nu}$ (eta, the Charlton metric) is the metric of the observer-split manifold. It is not flat space: it is deformed by the observer congruence ω.

The symbol $\delta_{\mu\nu}$ (the Kronecker delta) equals 1 when $\mu = \nu$ and 0 otherwise. It is the identity matrix in index notation—the "flat background" before observer splitting.

The operator $\otimes$ (tensor product, sometimes written as an outer product) means $(\omega\otimes\omega)_{\mu\nu} = \omega_\mu\,\omega_\nu$. The tensor product of ω with itself produces a rank-2 tensor from a rank-1 object.

The factor of 2 in the Charlton metric distinguishes it from the projector Π. The metric η has signature that depends on the norm of ω; for a unit timelike congruence $\omega_\mu\,\omega^\mu = 1$, the metric becomes $\eta = \mathrm{diag}(-1, +1, +1, +1)$ in adapted coordinates—a Lorentzian-like signature.

The tensor $\Pi_{\mu\nu}$ (Pi, the spatial projector) projects any tensor onto the subspace orthogonal to ω. It removes the temporal component, leaving only spatial content. In HUFD, Π ensures that the curvature seen by the semantic system lives entirely in the spatial submanifold.

Expansion — Projector Acting on Curvature

The projected curvature that enters the Constant-Attention update is:

$$\Pi\,R_{\mu\nu}\,\Pi \;=\; \left(\delta_\mu^{\ \alpha} - \omega_\mu\,\omega^\alpha\right) R_{\alpha\beta}\left(\delta_\nu^{\ \beta} - \omega^\beta\,\omega_\nu\right) \tag{14}$$

The raised index $\omega^\alpha = \eta^{\alpha\beta}\,\omega_\beta$ is the observer congruence with the index raised using the inverse Charlton metric. The Greek indices α, β are dummy summation indices contracted over.

This double projection removes any component of $R_{\mu\nu}$ aligned with the observer direction ω, yielding purely "spatial" curvature.

Implementation

```
import numpy as np

def charlton_metric(omega):
    """
    Compute eta = delta - 2 * omega (x) omega.
    omega : array of shape (4,) -- observer congruence 1-form
    """
    delta = np.eye(4)
    eta = delta - 2.0 * np.outer(omega, omega)
    return eta

def projector(omega):
    """
```

```
    Compute Pi = delta - omega (x) omega.
    """
    return np.eye(4) - np.outer(omega, omega)

def project_curvature(R, Pi):
    """
    Compute Pi . R . Pi  (double contraction).
    R  : array of shape (4, 4, 2, 2) -- curvature tensor
    Pi : array of shape (4, 4)        -- spatial projector
    """
    # Contract: (Pi)^alpha_mu  R_{alpha beta}  (Pi)^beta_nu
    R_proj = np.einsum('ma, abij, bn -> mnij', Pi, R, Pi)
    return R_proj
```

The Field Equation — Gradient Flow

Compact Form (Eq. 28)

$$\partial_t \mathcal{H}_\mu \;=\; -\,\frac{\delta\, S_{\mathrm{HUFD}}}{\delta\, \mathcal{H}_\mu} \tag{15}$$

Symbol Descriptions

The left-hand side $\partial_t \mathcal{H}_\mu$ (partial time derivative of the connection) is the rate of change of the field at each point. It is an $\mathfrak{su}(2)$-valued one-form, the same type of object as $\mathcal{H}_\mu$ itself.

The right-hand side $\delta\, S_{\mathrm{HUFD}}/\delta\, \mathcal{H}_\mu$ (functional derivative of the action with respect to the connection) is the HUFD analogue of a gradient. It asks: "if I change $\mathcal{H}_\mu$ by an infinitesimal amount at one point, how does the total action change?" The negative sign means the field flows downhill—toward lower action.

This equation is a gradient flow or steepest descent on the infinite-dimensional space of field configurations. Its solutions are trajectories that monotonically decrease S_{HUFD}.

First Expansion (Eq. 5.2)

Computing the functional derivative explicitly:

$$\partial_t \mathcal{H}_\mu \;=\; -\,\kappa\, \frac{\delta}{\delta\, \mathcal{H}_\mu}\Big(\alpha\, L_{\mathrm{sem}} + \beta\, \mathcal{E}_d + \gamma\, \Phi^{-1}\Big) \;-\; \lambda\, \frac{\delta}{\delta\, \mathcal{H}_\mu}\, \mathrm{Tr}\big(R_{\rho\sigma}\, R^{\rho\sigma}\big) \tag{16}$$

The coefficient κ (kappa, a positive real number) sets the timescale for the observable terms (lucidity, drift, phase-lock). The coefficient λ (lambda, positive) sets the timescale for the curvature term. Together they control the relative speed at which different parts of the action drive the evolution.

The indices ρ and σ (rho and sigma) are dummy summation indices distinct from μ, ν. They indicate that the curvature energy sums over *all* pairs of directions, not just the ones involving μ.

Second Expansion — Lattice Discretisation (Eq. 29)

On a discrete lattice with time step Δt and spatial projector Π:

$$\mathcal{H}^{(t+1)} \;=\; \mathcal{H}^{(t)} \;-\; \Delta t\Big(-\,\nabla^2 \mathcal{H}^{(t)} \;+\; g_{\mathrm{eff}}^2\, \mathcal{H}^{(t)}\Big) \;+\; \Pi\, \Xi(x_t) \tag{17}$$

The superscripts (t) and $(t+1)$ label discrete time steps on the lattice, replacing the continuous ∂_t with a finite difference.

The time step Δt (delta-t) is a small positive number controlling the granularity of the update. Smaller Δt gives more accuracy but requires more iterations to reach equilibrium.

The discrete Laplacian $\nabla^2 \mathcal{H}^{(t)}$ is the sum of the connection values at neighbouring lattice sites minus the central value, divided by the lattice spacing squared. It measures the local average of $\mathcal{H}$ relative to the value at a point—the "how different am I from my neighbours" operator.

The effective coupling g_{eff}^2 ("g-effective squared") is a mass-like stiffness parameter derived from the consensus potential. It controls how tightly the field is pinned to its equilibrium. Larger g_{eff}

means stiffer dynamics; the field snaps back quickly. Smaller g_{eff} means softer dynamics and longer-range memory.

The injection operator $\Xi(x_t)$ (Xi, evaluated at the current input x_t) introduces new information into the field. Every new input token, observation, or perturbation enters the system through this term. It is an $\mathfrak{su}(2)$-valued quantity computed from the input.

The projector Π multiplying Ξ ensures the injected information respects observer congruence—only the spatial (semantic) components of the new information enter the field, not temporal artefacts.

Third Expansion — Fully Explicit Lattice Step

On a lattice with nodes i and neighbours $\mathcal{N}(i)$:

$$\mathcal{H}_i^{(t+1)} = \mathcal{H}_i^{(t)} - \Delta t \left(- \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} (\mathcal{H}_j^{(t)} - \mathcal{H}_i^{(t)}) + g_{\text{eff}}^2 \, \mathcal{H}_i^{(t)} \right) + \Pi_i \, \Xi_i(x_t) \tag{18}$$

The set $\mathcal{N}(i)$ (the neighbourhood of node i) contains all lattice nodes directly connected to i. Its cardinality $|\mathcal{N}(i)|$ is the number of such neighbours.

The sum $\sum_{j \in \mathcal{N}(i)} (\mathcal{H}_j^{(t)} - \mathcal{H}_i^{(t)})$ is the discrete Laplacian: it computes how much the connection at node i differs from its neighbours. If $\mathcal{H}_i$ is much larger than its neighbours, this term is negative, pulling $\mathcal{H}_i$ down. If $\mathcal{H}_i$ is much smaller, it pushes $\mathcal{H}_i$ up. This is the diffusion mechanism that smooths the field.

Implementation

```
import numpy as np

def discrete_hufd_step(H, neighbors, g_eff, dt, Pi, Xi):
    """
    One lattice update step of the HUFD field equation.

    H         : array [N, 2, 2]   -- connection at each node (su(2)
        matrices)
    neighbors : list of lists      -- neighbors[i] = [j1, j2, ...]
        for node i
    g_eff     : float              -- effective coupling constant
    dt        : float              -- time step
    Pi        : array [N, 2, 2]    -- spatial projector at each node
    Xi        : array [N, 2, 2]    -- injection from current input
    """
    N = H.shape[0]
    H_new = np.copy(H)

    for i in range(N):
        nbrs = neighbors[i]
        n_nbrs = len(nbrs)
        if n_nbrs == 0:
            continue

        # Discrete Laplacian
        laplacian = sum(H[j] - H[i] for j in nbrs) / n_nbrs
```

```
        # Update: H^{t+1} = H^t - dt * (-laplacian + g_eff^2 * H^t) +
            Pi * Xi
        H_new[i] = H[i] - dt * (-laplacian + g_eff**2 * H[i]) + Pi[i]
            @ Xi[i]

    return H_new
```

The Lyapunov Stability Guarantee

Compact Form

$$\frac{d}{dt}\, S_{\mathrm{HUFD}} \;\leq\; 0 \tag{19}$$

Symbol Descriptions

The time derivative $d/dt\; S_{\mathrm{HUFD}}$ measures how fast the total action is changing. The inequality states that this quantity is never positive: the action either decreases or remains constant at every moment in time.

This is a Lyapunov condition. A Lyapunov functional is a scalar quantity that is bounded below and non-increasing under the dynamics. Its existence guarantees that the system cannot diverge, oscillate without bound, or enter chaotic trajectories. It must converge to a stable equilibrium or a limit cycle.

Expansion — Why It Holds

The proof follows directly from the gradient-flow structure of the field equation. Substituting Eq. (15) into the time derivative of the action:

$$\frac{d}{dt}\, S_{\mathrm{HUFD}} \;=\; \int d^4x\; \mathrm{Tr}\left(\frac{\delta\, S_{\mathrm{HUFD}}}{\delta\, \mathcal{H}_\mu}\, \partial_t\, \mathcal{H}_\mu\right) \;=\; -\int d^4x\; \mathrm{Tr}\left(\frac{\delta\, S_{\mathrm{HUFD}}}{\delta\, \mathcal{H}_\mu}\, \frac{\delta\, S_{\mathrm{HUFD}}}{\delta\, \mathcal{H}_\mu}\right) \;=\; -\int d^4x\, \left\| \frac{\delta\, S_{\mathrm{HUFD}}}{\delta\, \mathcal{H}_\mu} \right\|^2 \;\leq\; 0 \tag{20}$$

The first equality is the chain rule applied to the action functional. The second equality substitutes the field equation $\partial_t \mathcal{H}_\mu = -\delta S_{\mathrm{HUFD}}/\delta \mathcal{H}_\mu$. The third equality recognises the integrand as the squared norm of the functional gradient, which is always non-negative. The negative sign in front makes the entire expression non-positive.

The integrand $\|\delta S_{\mathrm{HUFD}}/\delta \mathcal{H}_\mu\|^2$ is the local "driving force" of the dynamics. Where this is large, the field is far from equilibrium and evolving quickly. Where it is zero, the field has reached a local minimum of the action. The integral vanishes (equality $d/dt\, S_{\mathrm{HUFD}} = 0$) if and only if the functional derivative is zero everywhere—the system has reached equilibrium.

Implementation

```python
def check_lyapunov(S_old, S_new, tol=1e-10):
    """
    Verify the Lyapunov condition: S_new <= S_old.

    Returns True if the condition holds (within numerical tolerance),
    along with the change dS.
    """
    dS = S_new - S_old
    condition_holds = dS <= tol
    return condition_holds, dS
```

The Gauge Symmetry

Compact Form (Eq. 3.1)

$$\mathcal{H}_\mu \longmapsto \mathcal{H}'_\mu = U\,\mathcal{H}_\mu\,U^{-1} - (\partial_\mu U)\,U^{-1} \tag{21}$$

$$\psi \longmapsto \psi' = U\,\psi \tag{22}$$

Symbol Descriptions

The map $\longmapsto$ denotes a gauge transformation—a change of internal reference frame at each point. The physics (action, curvature energy, all observables) is unchanged by this transformation; only the representation changes.

The matrix $U(x) \in \mathrm{SU}(2)$ (a local gauge transformation) is a unitary 2×2 matrix with determinant 1, varying smoothly from point to point. It acts on the internal degrees of freedom.

The conjugation $U\,\mathcal{H}_\mu\,U^{-1}$ rotates the connection into the new frame. The inverse U^{-1} is the matrix inverse of U; for SU(2), this equals the conjugate transpose $U^\dagger$.

The inhomogeneous term $(\partial_\mu U)\,U^{-1}$ compensates for the fact that U varies from point to point. Without this term, the covariant derivative would not transform correctly. This is the same structure as in Yang–Mills gauge theory.

Expansion — Curvature Transforms Covariantly

Under this transformation, the curvature transforms as:

$$R_{\mu\nu} \longmapsto R'_{\mu\nu} = U\,R_{\mu\nu}\,U^{-1} \tag{23}$$

The curvature rotates by conjugation—it does not pick up an inhomogeneous derivative term. This means the curvature "lives in the adjoint representation" of SU(2).

The trace of the squared curvature is therefore invariant:

$$\mathrm{Tr}(R'_{\mu\nu}\,R'^{\mu\nu}) = \mathrm{Tr}(U\,R_{\mu\nu}\,U^{-1}\,U\,R^{\mu\nu}\,U^{-1}) = \mathrm{Tr}(R_{\mu\nu}\,R^{\mu\nu}) \tag{24}$$

The second equality follows from the cyclic property of the trace ($\mathrm{Tr}(ABC) = \mathrm{Tr}(CAB)$) and the cancellation $U^{-1}\,U = \mathbb{1}$ (the identity matrix).

This guarantees that the HUFD action is gauge-invariant: physically distinct configurations are distinguished by gauge-invariant quantities (curvature, holonomy, observables), not by the choice of internal frame.

Implementation

```
import numpy as np
from scipy.linalg import expm

def gauge_transform_connection(H_mu, U, dU_mu):
    """
    Apply gauge transformation to connection.
    H_mu  : array (2,2) -- connection component
```

```
    U      : array (2,2) -- SU(2) gauge element
    dU_mu : array (2,2) -- partial derivative of U in mu direction
    """
    U_inv = np.conj(U).T  # for SU(2), inverse = conjugate transpose
    return U @ H_mu @ U_inv - dU_mu @ U_inv

def gauge_transform_curvature(R, U):
    """Curvature transforms by conjugation."""
    U_inv = np.conj(U).T
    return U @ R @ U_inv

def random_su2():
    """Generate a random SU(2) element via exponential map."""
    # Random element of su(2) algebra
    coeffs = np.random.randn(3)
    algebra_elem = sum(c * 1j * sigma[a] / 2
                       for a, c in enumerate(coeffs))
    return expm(algebra_elem)
```

The Constant-Attention Update

Compact Form (Eq. 2.2)

$$\mathcal{H}_t \;=\; \Phi\big(\mathcal{H}_{t-1},\; \Pi\, R_{\mu\nu}\, \Pi,\; L_{\text{sem}}\big) \tag{25}$$

Symbol Descriptions

The subscript t on $\mathcal{H}_t$ is a discrete time index (not a spacetime component index). $\mathcal{H}_t$ is the connection at time step t; $\mathcal{H}_{t-1}$ is its value at the previous step.

The function Φ here denotes the update map—not the phase-lock index from the action. (The paper overloads this symbol; context distinguishes them.) It takes three arguments: the previous state, the projected curvature, and the lucidity.

The projected curvature $\Pi\, R_{\mu\nu}\, \Pi$ is the spatial curvature with the observer direction removed, as defined in Section 3.

The key property of this update is that it is *path-independent* (holonomic). The result of applying the update does not depend on the order in which inputs arrived, only on the total curvature and lucidity accumulated. This is what "Constant Attention" means: the field maintains a persistent coupling to the global holonomic state rather than recomputing attention weights at each step.

Expansion — The Constant-Attention Loop

The update decomposes into the five-stage loop described in the HUFD architecture:

$$\mathcal{H}_t \;=\; \mathcal{H}_{t-1} \underbrace{-\,\Delta t\,\big(-\nabla^2\mathcal{H}_{t-1} + g_{\text{eff}}^2\,\mathcal{H}_{t-1}\big)}_{\text{diffusion + stiffness}} \underbrace{+\,\Pi\,\Xi(x_t)}_{\text{injection}} \tag{26}$$

followed by SOM validation (checking quantisation error $QE = \|x - w_{\text{BMU}}\|$, where w_{BMU} is the weight vector of the best-matching unit on the SOM grid) and RAGU feedback (gauge-covariant retrieval update with cost proportional to $\log N$, where N is the graph size).

The quantisation error QE (a non-negative scalar) measures how well the current state matches the nearest archetype in the SOM. Lower QE means the state is close to a known attractor; higher QE signals novelty or drift.

Implementation

```
def constant_attention_step(H_prev, neighbors, g_eff, dt, Pi, Xi,
                            som, x_input):
    """
    One step of the Constant-Attention loop:
    1. Field update (diffusion + stiffness + injection)
    2. SOM validation
    3. Return updated state with diagnostics
    """
    # Step 1: Field update (Eq. 29)
    H_new = discrete_hufd_step(H_prev, neighbors, g_eff, dt, Pi, Xi)

    # Step 2: SOM validation
    som_state = som.project(x_input)
    qe = som_state.q_error
```

```
    lucidity = som_state.lucidity

    # Step 3: Modulate update by lucidity (RAGU feedback)
    # High lucidity -> accept update fully
    # Low lucidity  -> dampen update, stay closer to previous state
    modulation = np.clip(lucidity, 0.1, 1.0)
    H_final = H_prev + modulation * (H_new - H_prev)

    return H_final, {'qe': qe, 'lucidity': lucidity}
```

Lattice Holonomy and Discrete Curvature

Compact Form

$$H_F \;=\; P\exp\oint_{\partial F}\mathcal{H}_\mu\,dx^\mu \;\xrightarrow{\text{lattice}}\; \prod_{e\in\partial F} U_e \tag{27}$$

Symbol Descriptions

The holonomy H_F (a group element in SU(2)) is the result of parallel-transporting a state around the closed boundary of face F. It measures the total curvature enclosed by that loop.

The symbol P (script-P, path ordering) is an operator that orders the exponential so that factors appear in the sequence dictated by the path of integration. Without path ordering, the exponential of a non-commutative quantity is ill-defined.

The contour integral $\oint_{\partial F}$ runs around the boundary of face F.

On the lattice, the path-ordered exponential becomes a finite product of SU(2) group elements U_e, one per edge e in the boundary of F, taken in the order the boundary is traversed.

Each group element $U_e \in \mathrm{SU}(2)$ is a unit quaternion (w, x, y, z) with $w^2 + x^2 + y^2 + z^2 = 1$. It represents the transport of meaning along a single edge of the lattice. The relationship between the Lie-algebra connection $\mathcal{H}_\mu$ and the group element U_e is $U_e = \exp(\mathcal{H}_\mu\,\Delta x^\mu)$ for a short edge of length Δx.

Expansion — Quaternion Product

For a triangular face with edges AB, BC, CA:

$$H_F \;=\; U_{AB}\;\cdot\;U_{BC}\;\cdot\;U_{CA} \tag{28}$$

Each quaternion product is computed by Hamilton's rule. For two unit quaternions $q_1 = (a, b, c, d)$ and $q_2 = (e, f, g, h)$:

$$q_1 \cdot q_2 \;=\; \begin{pmatrix} ae - bf - cg - dh \\ af + be + ch - dg \\ ag - bh + ce + df \\ ah + bg - cf + de \end{pmatrix} \tag{29}$$

where the four rows give the (w, x, y, z) components respectively. This rule derives from the defining relations of the quaternion algebra: $\mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 = \mathbf{ijk} = -1$.

Curvature from Holonomy

The discrete curvature of face F is extracted from the holonomy:

$$\kappa_F \;=\; 2\,\arccos\!\left|w_{H_F}\right| \tag{30}$$

where w_{H_F} is the scalar (real) component of the holonomy quaternion H_F. This gives the rotation angle in radians. When $\kappa_F = 0$, the face is flat (identity holonomy, perfect coherence). When $\kappa_F = \pi$, the face has maximum curvature (a 180° rotation, maximum contradiction).

Relaxation — SLERP Correction

Each edge is nudged toward flatness by:

$$U_e^{(\text{new})} \;=\; U_e \;\cdot\; \text{slerp}\Big(\mathbb{1},\; H_F^{-1},\; \frac{\alpha}{n_{\text{links}}}\Big) \tag{31}$$

The identity quaternion $\mathbb{1} = (1, 0, 0, 0)$ represents zero rotation. The inverse holonomy H_F^{-1} is the conjugate quaternion $(w, -x, -y, -z)$ for a unit quaternion—the correction that would completely flatten the face in one step.

The SLERP (Spherical Linear Interpolation) between $\mathbb{1}$ and H_F^{-1} at parameter $t = \alpha/n_{\text{links}}$ gives a quaternion that is fraction t of the way from identity to the full correction:

$$\text{slerp}(q_1,\, q_2,\, t) \;=\; q_1\,\frac{\sin\big((1-t)\,\theta\big)}{\sin\theta} \;+\; q_2\,\frac{\sin(t\,\theta)}{\sin\theta} \tag{32}$$

where $\theta = \arccos(q_1 \cdot q_2)$ is the angle between the two quaternions on the unit 3-sphere.

The parameter α (step size) is a small positive number controlling how aggressively the correction is applied. The division by n_{links} (the number of edges in the face) distributes the correction equally among all edges.

Implementation

```
import numpy as np

def quat_multiply(q1, q2):
    """
    Multiply two quaternions q1 * q2.
    Each quaternion is (w, x, y, z).
    """
    a, b, c, d = q1
    e, f, g, h = q2
    return np.array([
        a*e - b*f - c*g - d*h,
        a*f + b*e + c*h - d*g,
        a*g - b*h + c*e + d*f,
        a*h + b*g - c*f + d*e
    ])

def quat_conjugate(q):
    """Inverse of a unit quaternion: negate vector part."""
    return np.array([q[0], -q[1], -q[2], -q[3]])

def holonomy(edges):
    """
    Compute holonomy = product of transport quaternions around a face
        .
    edges : list of arrays, each shape (4,) -- quaternions in
        traversal order
    """
    H = np.array([1.0, 0.0, 0.0, 0.0])  # identity
    for U_e in edges:
        H = quat_multiply(H, U_e)
    return H
```

```python
def curvature_from_holonomy(H):
    """Extract scalar curvature kappa = 2 * arccos(|w|)."""
    return 2.0 * np.arccos(np.clip(np.abs(H[0]), 0.0, 1.0))

def slerp(q1, q2, t):
    """
    Spherical linear interpolation between q1 and q2 at parameter t.
    """
    dot = np.clip(np.dot(q1, q2), -1.0, 1.0)
    # If quaternions are nearly equal, use linear interpolation
    if np.abs(dot) > 0.9995:
        result = q1 + t * (q2 - q1)
        return result / np.linalg.norm(result)
    theta = np.arccos(np.abs(dot))
    sin_theta = np.sin(theta)
    s1 = np.sin((1 - t) * theta) / sin_theta
    s2 = np.sin(t * theta) / sin_theta
    # Ensure shortest path
    if dot < 0:
        q2 = -q2
    return s1 * q1 + s2 * q2

def relaxation_step(edges, face_edges, alpha=0.1):
    """
    One relaxation step: correct each edge toward flattening its
       faces.
    edges      : dict {edge_id: quaternion}
    face_edges : list of lists -- each face is [edge_id_1, edge_id_2,
        ...]
    """
    corrections = {eid: [] for eid in edges}
    identity = np.array([1.0, 0.0, 0.0, 0.0])

    for face in face_edges:
        # Compute holonomy of this face
        H = holonomy([edges[eid] for eid in face])
        H_inv = quat_conjugate(H)
        n_links = len(face)
        t = alpha / n_links

        # SLERP correction for each edge
        correction = slerp(identity, H_inv, t)
        for eid in face:
            corrections[eid].append(correction)

    # Apply averaged corrections
    new_edges = {}
    for eid, q in edges.items():
        if corrections[eid]:
            # Average all face corrections for this edge
            avg_corr = np.mean(corrections[eid], axis=0)
            avg_corr /= np.linalg.norm(avg_corr)
            new_edges[eid] = quat_multiply(q, avg_corr)
        else:
            new_edges[eid] = q
    return new_edges
```

The Observables

Attractor Entropy (Eq. A6)

$$S_A \;=\; -\sum_i p_i \log p_i \tag{33}$$

The attractor entropy S_A is the Shannon entropy computed over the SOM basins. The index i runs over all neurons (attractor basins) in the self-organising map. The probability p_i is the fraction of input states that map to basin i under the SOM's best-matching-unit assignment. The logarithm log is taken in natural (base-e) units, giving entropy in nats. High S_A means states are spread across many basins (diverse, exploratory). Low S_A means states are concentrated in few basins (focused, converged).

Under the HUFD lucidity flow, S_A decreases monotonically while the phase-lock index Φ increases, until a holonomically stable state is reached.

Total Curvature Energy

$$E_{\text{curv}} \;=\; \sum_F \kappa_F^2 \;=\; \sum_F \Big(2\,\arccos\!|w_{H_F}|\Big)^2 \tag{34}$$

The sum runs over all faces F in the lattice. This is the discrete analogue of $\mathrm{Tr}(R_{\mu\nu}\,R^{\mu\nu})$ from the continuum action. The Law of Cognitive Conservation states that this quantity is non-increasing under natural dynamics:

$$\frac{d}{dt}\,\mathrm{Tr}(R_{\mu\nu}\,R^{\mu\nu}) \;\le\; 0 \tag{35}$$

Lucidity Flux

$$D_\mu\,J^\mu_{\text{hol}} \;=\; 0 \tag{36}$$

The holonomic current J^μ_{hol} (the lucidity flux) is a conserved vector field. The covariant divergence $D_\mu\,J^\mu = 0$ states that the total lucidity flux is conserved—meaning neither created nor destroyed, only redistributed. This is the Constant-Attention conservation law. The circulation of the connection around any closed loop is constant:

$$\oint_C \mathcal{H}_\mu\,dx^\mu \;=\; \text{const.} \tag{37}$$

Implementation

```
import numpy as np

def attractor_entropy(som_assignments, n_basins):
    """
    Compute Shannon entropy over SOM basin assignments.
    som_assignments : array of int -- basin index for each input
        state
    n_basins        : int          -- total number of SOM neurons
    """
    counts = np.bincount(som_assignments, minlength=n_basins)
```

```
    probs = counts / counts.sum()
    # Remove zeros to avoid log(0)
    probs = probs[probs > 0]
    return -np.sum(probs * np.log(probs))

def total_curvature_energy(edges, face_edges):
    """
    Compute sum of squared curvatures over all faces.
    """
    E = 0.0
    for face in face_edges:
        H = holonomy([edges[eid] for eid in face])
        kappa = curvature_from_holonomy(H)
        E += kappa ** 2
    return E
```

Summary of Rewrite Rules

The following table collects every transformation used in the expansion chains above. Each entry is a named rewrite rule: a pattern on the left is replaced by the expansion on the right. These are the rules that fire, one at a time, when the user steps through a computation in the Live Equation Rewriter.

Rule Name	**Transformation**
Curvature Definition	$R_{\mu\nu} \longrightarrow [\mathcal{H}_\mu,\, \mathcal{H}_\nu] + \omega_{[\mu}\eta_{\nu]}$
Commutator Expansion	$[\mathcal{H}_\mu,\, \mathcal{H}_\nu] \longrightarrow \mathcal{H}_\mu\mathcal{H}_\nu - \mathcal{H}_\nu\mathcal{H}_\mu$
Pauli Basis Expansion	$\mathcal{H}_\mu \longrightarrow \sum_a \mathcal{H}^a_\mu\, (i\sigma_a/2)$
Component Curvature	$[\mathcal{H}_\mu, \mathcal{H}_\nu]^c \longrightarrow -\epsilon_{abc}\, \mathcal{H}^a_\mu\, \mathcal{H}^b_\nu$
Lattice Discretisation	$\partial_t \mathcal{H}_\mu = -\delta S_{\mathrm{HUFD}}/\delta\mathcal{H}_\mu \longrightarrow \mathcal{H}^{(t+1)} = \mathcal{H}^{(t)} - \Delta t(\cdots) + \Pi\Xi$
Holonomy Product	$H_F = P \exp \oint \mathcal{H}_\mu\, dx^\mu \longrightarrow \prod_e U_e$
Value Substitution	$U_e \longrightarrow (w, x, y, z)$ from the data table
Quaternion Multiplication	$(a, b, c, d) \cdot (e, f, g, h) \longrightarrow$ Hamilton's rule (Eq. 29)
Curvature Extraction	$H_F = (w, x, y, z) \longrightarrow \kappa_F = 2 \arccos \lvert w \rvert$
SLERP Correction	$U_e \longrightarrow U_e \cdot \mathrm{slerp}(\nVdash,\, H_F^{-1},\, \alpha/n)$
Lyapunov Check	$\sum_F \kappa_F^2\big\vert_{\mathrm{new}} \leq \sum_F \kappa_F^2\big\vert_{\mathrm{old}}$
Gauge Transformation	$\mathcal{H}_\mu \longrightarrow U\mathcal{H}_\mu U^{-1} - (\partial_\mu U)U^{-1}$
Euler–Lagrange	$\delta S_{\mathrm{HUFD}}/\delta\mathcal{H}_\mu = 0 \longrightarrow$ field equation (Eq. 16)
Gradient Flow	$\partial_t \mathcal{H}_\mu = -\delta S_{\mathrm{HUFD}}/\delta\mathcal{H}_\mu$

Each of these rules has a precise scope: it fires only when its left-hand pattern appears in the current expression, and it produces exactly the right-hand replacement. The user who has stepped through these rules enough times will recognise them by name and be able to predict the next transformation before seeing it. This is the pedagogical mechanism of the Live Equation Rewriter: the rewrite rules *are* the mathematics, made visible and interactive.